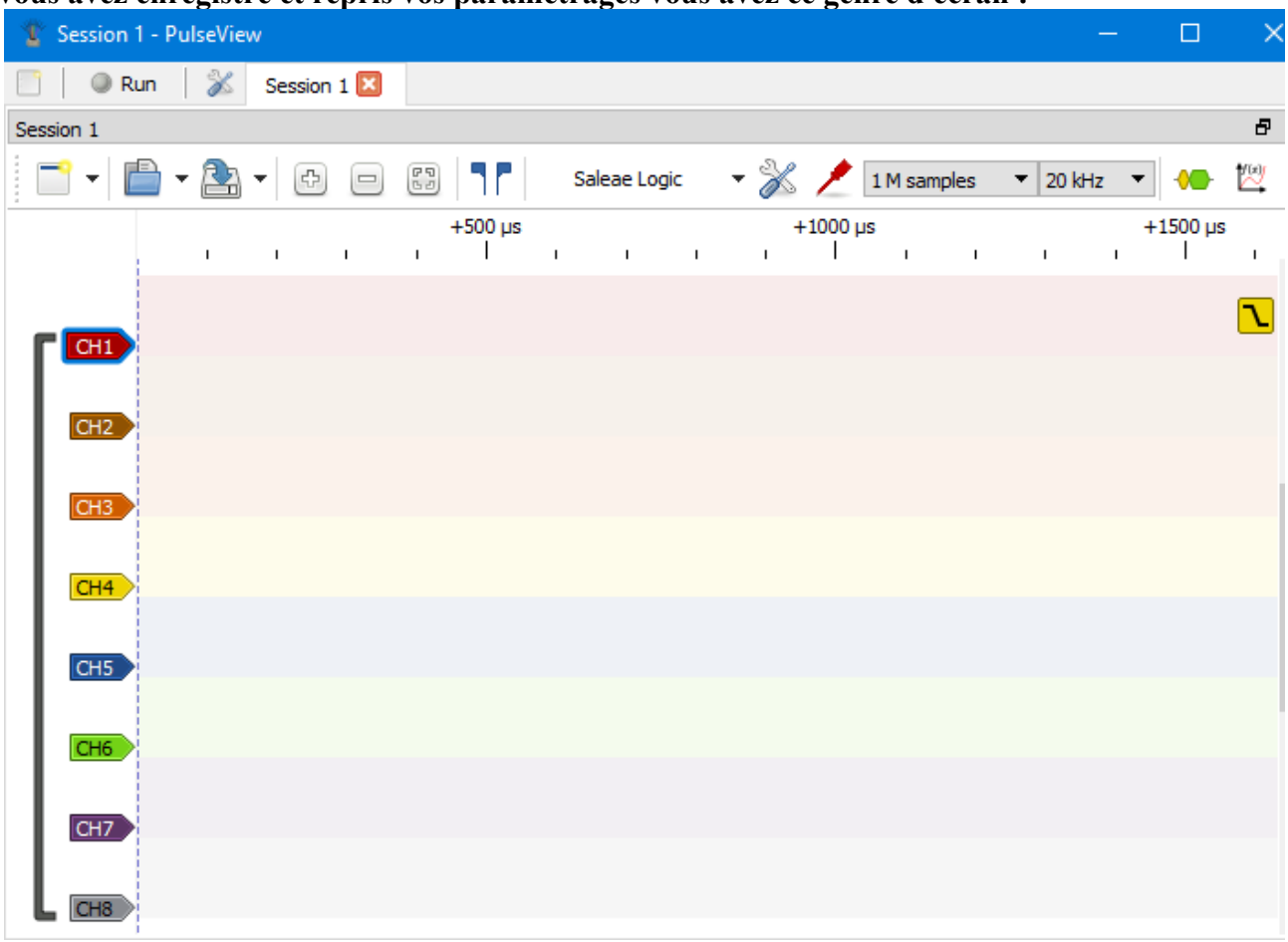
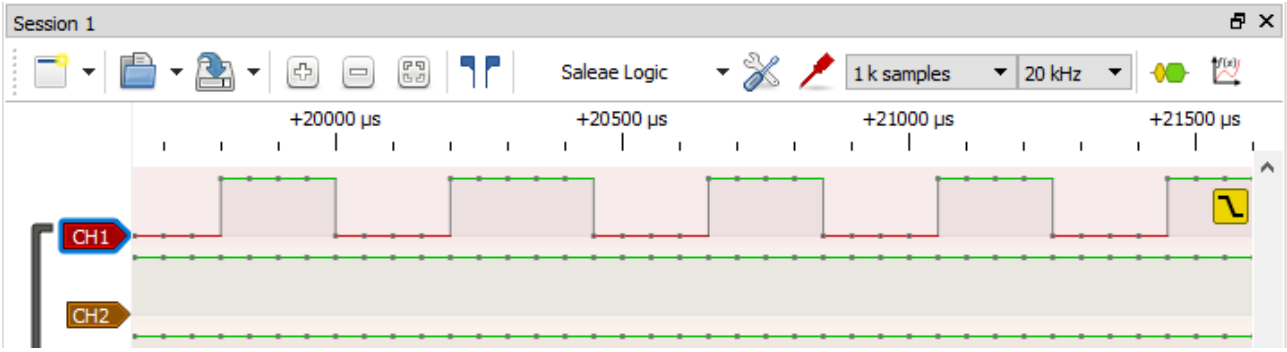
USART et analyseur logique

On doit avoir suivi le tutoriel de prise en main de l'analyseur logique avant.
Attention PulseView doit être lancé et l analyseur logique branché avant de connecter la carte Nucleo sur le port USB sous peine de ne pas détecter l'analyseur logique

si vous avez enregistré et repris vos paramétrages vous avez ce genre d'écran :



Il faut régler la voie sur laquelle on va brancher le signal émis par la liaison série sur Front descendant (voir tutoriel 1)
On choisit une fréquence d'échantillonnage compatible avec notre baudrate pour réussir à avoir la trame complète : si je veux générer du 9600Bauds et que je veux mesurer ce baudrate à 1% près, il faut que j'échantillonne à une fréquence au moins 100 fois supérieur à 9600Hz



si j'échantillonne pas assez haut en fréquence, je n'ai pas assez de point par niveau logique pour mesurer précisément sa durée : un signal périodique ne semble plus régulier ….ici parfois 5 ou 6

```
// compléter ce code à trous pour pouvoir tester l'usart2
void init_usart2(void)
{     //1. Activer l'horloge du port GPIOA et USART2

      // 2. PX.Y1 = USARTn_TX : Alternate Function Push-Pull, ?? MHz

         // 3. PX.Y2 = USARTn_RX : Input floating pull up

        // 4. Baudrate = ?? bauds
           //    Horloge_peripherique = ?? MHz
           //    USARTDIV = Horloge_peripherique / (16 * ???) = xxx.yyyy
           //    BRR =     ;  //BRR contient la partie entiere xxx <<4  et la partie (int)(0.yyyy * 16)
   USART2->BRR = 7500; //calculé avec 9600 et 72MHz comme horloge périphérique
// USART_CR1_SBK                Send Break
// USART_CR1_RWU                 Receiver wakeup
// USART_CR1_RE                Receiver Enable
// USART_CR1_TE                Transmitter Enable
// USART_CR1_IDLEIE                IDLE Interrupt Enable
// USART_CR1_RXNEIE                 RXNE Interrupt Enable
// USART_CR1_TCIE                Transmission Complete Interrupt Enable
// USART_CR1_TXEIE                PE Interrupt Enable
// USART_CR1_PEIE                PE Interrupt Enable
// USART_CR1_PS                Parity Selection
// USART_CR1_PCE                Parity Control Enable
// USART_CR1_WAKE                Wakeup method
// USART_CR1_M                Word length
// USART_CR1_UE                USART Enable
// USART_CR1_OVER8                 USART Oversmapling 8-bits
   USART2->CR1 = ;
// USART_CR2_ADD                Address of the USART node
// USART_CR2_LBDL                LIN Break Detection Length
// USART_CR2_LBDIE                LIN Break Detection Interrupt Enable
// USART_CR2_LBCL                Last Bit Clock pulse
// USART_CR2_CPHA                Clock Phase
// USART_CR2_CPOL                Clock Polarity
// USART_CR2_CLKEN                 Clock Enable
// USART_CR2_STOP                STOP[1:0] bits (STOP bits)
// USART_CR2_STOP_0                Bit 0
// USART_CR2_STOP_1                Bit 1
// USART_CR2_LINEN                LIN mode enable
  USART2->CR2 =;
```

```
//USART_CR3_EIE                 Error Interrupt Enable */
//USART_CR3_IREN                 IrDA mode Enable */
//USART_CR3_IRLP                IrDA Low-Power */
//USART_CR3_HDSEL                 Half-Duplex Selection */
//USART_CR3_NACK                 Smartcard NACK enable */
//USART_CR3_SCEN                Smartcard mode enable */
//USART_CR3_DMAR                 DMA Enable Receiver */
//USART_CR3_DMAT                 DMA Enable Transmitter */
//USART_CR3_RTSE                RTS Enable */
//USART_CR3_CTSE                CTS Enable */
//USART_CR3_CTSIE                CTS Interrupt Enable */
//USART_CR3_ONEBIT                 One Bit method */
  USART2->CR3 =;
//USART_GTPR_PSC                 PSC[7:0] bits (Prescaler value) */
//USART_GTPR_GT                 Guard time value */
  USART2->GTPR =;
}
void init_proc(void)  {init_usart2(); }
#define REG_STATUS _____
#define FLAG_TXDISPO _____
#define ENVOI_POSSIBLE  ((REG_STATUS & FLAG_TXDISPO)!=0)
int main(void)
{volatile uint32_t cptdelay=0;
        init_proc();
        while(1)
        { if (ENVOI_POSSIBLE)        {  USART2->DR = 0x55; }
                for(cptdelay=0; cptdelay<1000;cptdelay++) ;
        }
}
```

Attention ce code ne compile pas, il faut le corriger et boucher les trous...

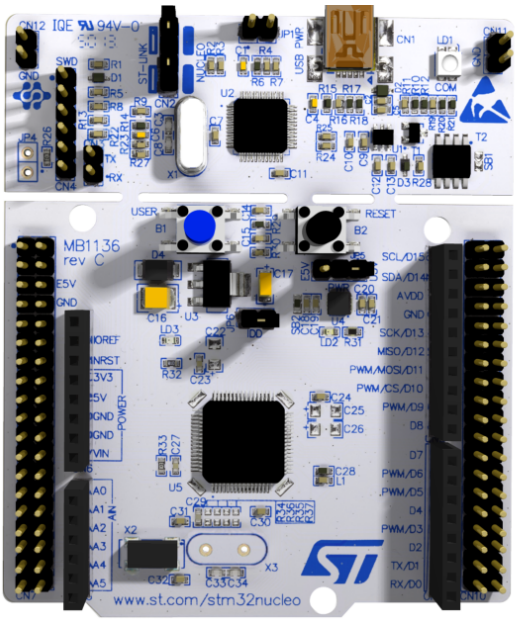
On envoie l'octet 0x55 en boucle, pour avoir une oscillation régulière sur TX (PA2)

On va espionner la trame : start = 0, B0=1, B1=1, …. B7=0, STOP=1

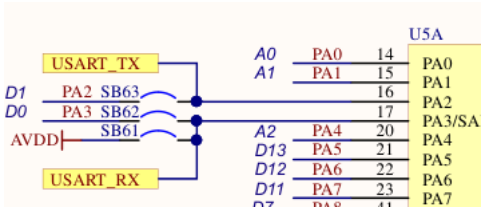
Si on ne met pas d'attente entre deux envois consécutifs alors on a une oscillation régulière.

Ce serait une fréquence moitié du baudrate (une période d oscillation correspond à 2 bits transmis : 1 puis 0 (ou 0 puis 1))

0n câble l'analyseur logique sur la nucléo en utilisant la documentation pour pouvoir espionner PA2



Attention au piège vu dans le tutoriel numéro 1...
Un coup d'oeil sur la documentation de la Nucléo montre que les signaux PA2 et PA3 sur le connecteur CN10 ne sont disponibles que si les Jumps SB63 et SB62 sont soudés.

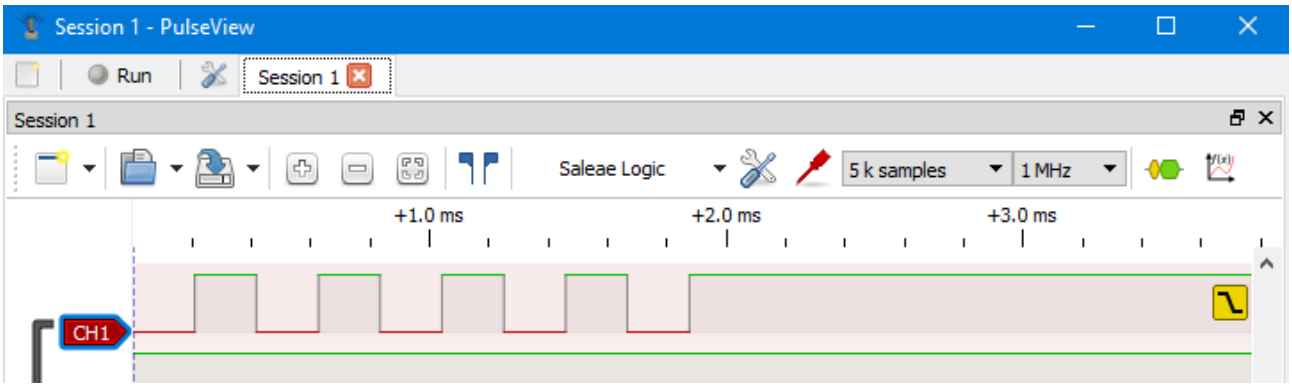


Un coup d'oeil sous la carte indique que non...

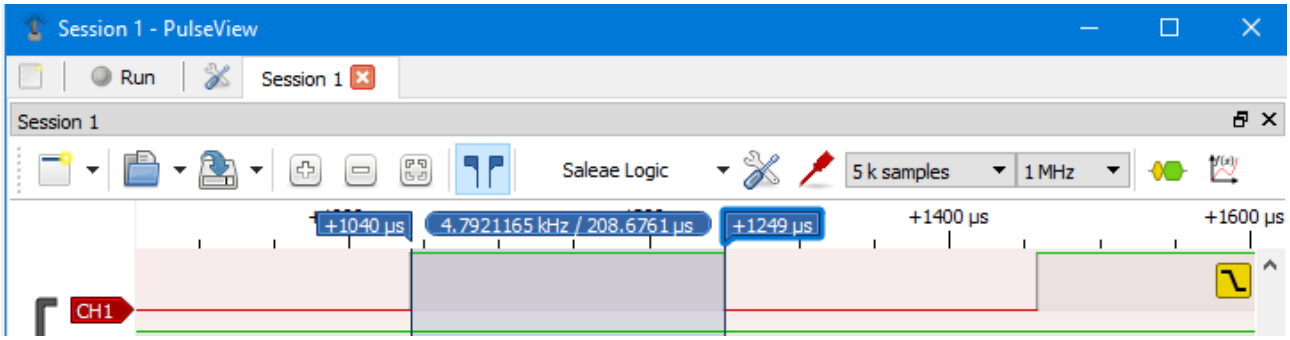
On se branche donc sur les picots RX et TX disponible dans la partie STLINK

Attention c est le RX du STLINK qui correspond au USART_TX

Si votre configuration est correcte, vous observez



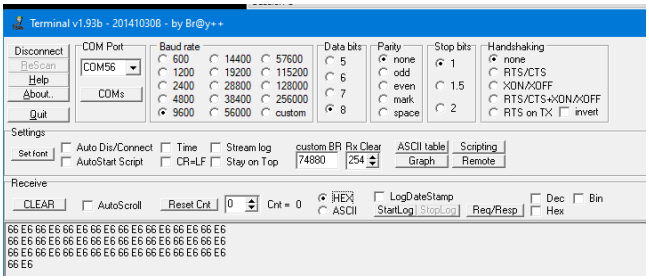
en utilisant le zoom et les marqueurs de mesure, on peut maintenant vérifier le baudrate :



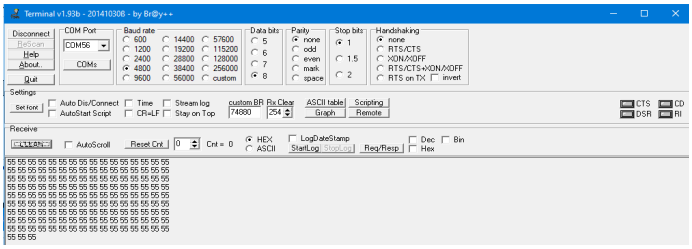
la durée pour émettre un bit correspond à une fréquence de 4.800 Khz, on a une erreur de réglage du baudrate...

IL FAUT ANALYSER ET COMPRENDRE POURQUOI...

si on teste avec un logiciel de reception série tel que Putty ou Terminal Bray, on arrive pas à recevoir la séquence ASCII 'U' 'U' …., en basculant en Hex, on observe la chose suivante :



0x66 puis 0XE6 au lieu de recevoir une série de valeurs 0x55, ce que l'on observe bien à 4800 bauds :



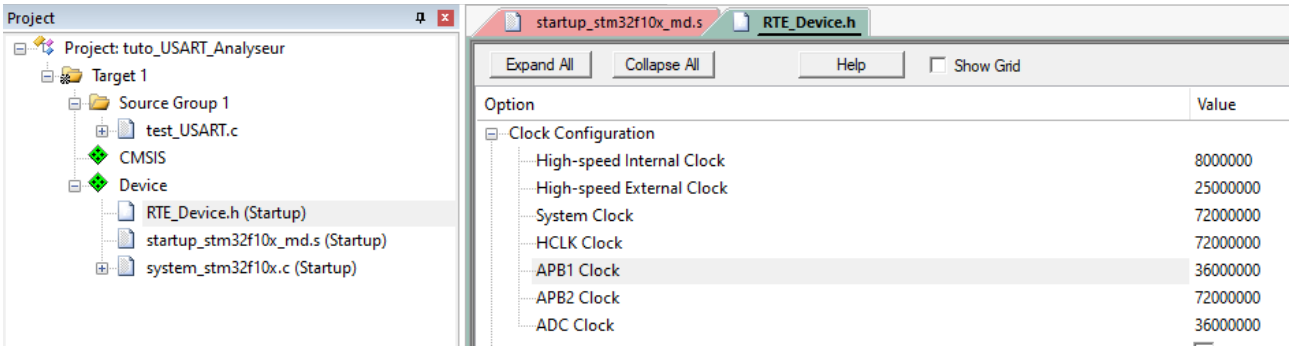
D'où vient l'erreur ?

Savoir mesurer le baudrate à l'analyseur logique est indispensable car il est difficile de deviner le vrai baudrate au vu de la séquence erronée.

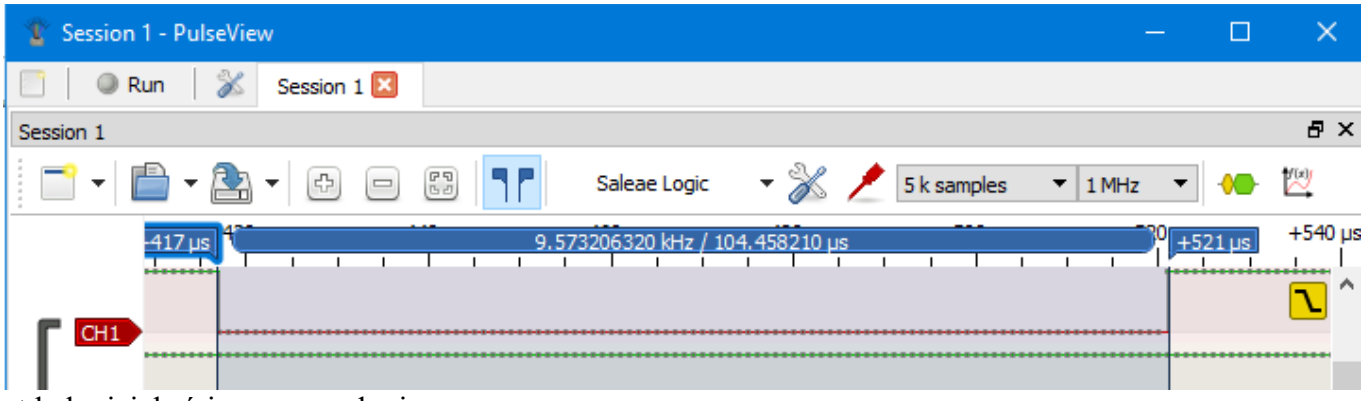
On a utilisé la formule 72 000 000 / 9600 /16 pour calculer la valeur de BR pour tomber sur un nombre à virgule : 468,75

la partie entiere est décalée de 4 bits vers la gauche. La partie après la virgule est multipliée par 16 et ajoutée.... en gros on peut charger directement dans BR la valeur 72 000 000 / 9600

l erreur vient certainement de l horloge présumée... UART2 dépend de APB1 et non pas de APB2



Le calcul du baudrate est erroné d'un facteur 2 , en reprenant la formule avec 36000000, on visualise la bonne vitesse de communication :



et le logiciel série comprend mieux...

